CrossAudit: A Git-Native, Cross-Vendor Audit Loop for Agentic Science

Zhaohe Dong
University of Cambridge
zd314@cam.ac.uk

Yuhao Chen
University of Wisconsin–Madison
ychen2546@wisc.edu

July 2026

Abstract

Autonomous "AI scientist" systems increasingly generate, execute, and review research. In the frontier systems we surveyed, the reviewing agent is typically an internal critic selected by the same operator and often drawn from the same model family or vendor as the generating agent. This arrangement leaves the supervision process vulnerable to self-preference bias and to blind spots shared across a common training pipeline; the supervision trace, moreover, usually lives in opaque platform logs rather than in an artefact a third party can replay. We present **CrossAudit**, a lightweight, infrastructure-minimal protocol for supervising autonomous research pipelines, built on three commitments. First, heterogeneity: every experiment increment produced by a generator agent is audited by an agent from a different model vendor, against a versioned, human-authored rulebook (the Constitution). Second, a git-native ledger: experimental increments, audit reports, verdicts, disputes, and escalations are all recorded as commits, allowing the supervision history to be revisited, compared, and cited by third parties. Third, graded human oversight: deterministic, model-free checks and clear inconsistencies can block the pipeline, whereas matters of judgement cannot; unresolved blockers are instead passed through a bounded revision loop and, if necessary, escalated to a human principal.

We define the protocol through eight invariants and describe a public reference implementation built with GitHub Actions and a few hundred lines of dependency-light Python. The implementation *targets* all eight invariants but currently enforces only a subset, including deterministic checks, reply validation, receipt generation, and anchored callbacks; receipt-verified, fail-closed admission is specified but not yet enforced. We also report a live deployment of a closely related variant in a computational-chemistry pipeline, in which the Claude-based generator, GPT-lineage auditor, and cloud-HPC compute layer are mutually decoupled.

We conducted an exploratory seeded-defect trial comprising 30 synthetic increments and 43 defects. The intended blinding was subsequently invalidated by a cross-vendor audit of our own repository; we accepted its findings and committed them alongside the trial data. Under the frozen scoring scheme, the same-family auditor identified 38 of 43 defects at both the lenient and strict tiers, with no false-positive blockers. The cross-vendor auditor identified 41 of 43 defects at the lenient tier and 37 of 43 at the strict tier, but returned `BLOCKED` on all 30 increments. Because recall under the frozen scorer increases with finding volume, and the three arms produced 17, 63, and 113 findings respectively, permutation-corrected agreement places the two model arms within noise at the lenient tier and reverses their ordering at the strict tier.

The trial therefore demonstrates that vendors can interpret the same rulebook differently; it does not establish that either auditor is superior. The strongest evidence in the repository is instead the committed record of cross-vendor audits applied to this paper itself. We argue that audit trails of this kind provide a practical basis for accountable machine science and can be adopted by an individual researcher using existing infrastructure.

Keywords: agentic science; AI scientist; scalable oversight; cross-vendor auditing; LLM-as-a-judge; self-preference bias; research integrity; human-in-the-loop.

1 Introduction

Two things motivated this paper. The first is the arrival of genuinely agentic research systems: pipelines in which large language model (LLM) agents propose hypotheses, write and execute code, run experiments, and draft manuscripts with limited human intervention. Sakana's AI Scientist produced machine-generated papers end-to-end [1, 2] and the line of work has since been published in *Nature* [3]; DeepMind's AI co-scientist organises hypothesis generation as a multi-agent tournament [4]; autonomous laboratories close the loop through to physical synthesis [8, 9]. Publishers and funders are already being urged to respond institutionally [12].

The second is older and more familiar: science's continuing struggle with reliability. In one large survey, most researchers had failed to reproduce a published result [11]. Agentic pipelines raise the stakes on both sides of this ledger. On one side, they can enforce a discipline that no human sustains: every increment logged, every parameter recorded. On the other, they can mass-produce plausible, well-formatted, wrong results faster than any human audit culture can absorb.

The relevant question, then, is not whether machine-generated science should be supervised, but how that supervision should be organised and who should provide it. Current frontier systems typically place supervision within the same architecture, using reflection agents, automated reviewers, or ranking tournaments [1, 4]. This creates a structural weakness: the critic and the creator are usually instantiated from the same model family or, at best, from models developed within the same vendor's training pipeline. LLM evaluators are known to favour their own generations, with self-recognition identified as a causal factor in this self-preference bias [5]; position and stylistic biases introduce further distortions [6]. Agents trained on similar data may also share systematic blind spots, so a reviewer with the same blind spots as the author may fail to detect the author's characteristic errors. At the same time, the supervision record (what was flagged, what was revised, and what was allowed to pass) often remains confined to transient context windows or proprietary platform logs. As a result, the communities asked to trust the resulting science cannot independently inspect the process by which it was reviewed.

Contributions. We make four contributions.

1. We articulate the *same-source supervision problem* in agentic science: internal critique by same-vendor models is structurally exposed to correlated failure (§2).
2. We specify **CrossAudit**, a supervision protocol defined by eight invariants (vendor heterogeneity, ledger completeness, citation validity, determinism-first, bounded revision, graded interruption, receipt binding, fail-closed admission) that together specify a re-inspectable, third-party-auditable (up to self-declared vendor identity, §5) supervision history (§3).
3. We describe a reference implementation requiring no infrastructure beyond two git repositories, a CI service, and two model API keys, together with a live deployment of a closely related variant in the first author's computational-chemistry pipeline, and we report what a second, product-oriented implementation of the same protocol taught us about the specification (§4).
4. We give an explicit threat model, including the attacks the protocol does *not* withstand, and situate the design against plausible alternatives (§5–§6).

CrossAudit is deliberately limited in what it claims to establish. It does not certify scientific truth; no textual audit can do so. Instead, it certifies a narrower and directly inspectable property: that each increment in a machine-generated research record was evaluated against a named, versioned set of rules by a configured auditor from a second vendor, and that the resulting decision and exchange were preserved in the ledger. The auditor has no write access to the increment's repository, although its vendor identity is currently self-declared (§5). The ledger is public wherever the operator chooses to publish it; at present, it preserves parsed audit reports, while capture of the raw model requests and responses is planned for revision 2. We argue that this narrower guarantee is an appropriate first target for accountable machine science, much as continuous integration, rather than formal verification, became the principal quality-control mechanism in modern software development.

2 Related work and the same-source problem

Agentic science systems. The AI Scientist [1, 2, 3] automates the ideation–experiment–writeup–review cycle, including an internal automated reviewer scored against human reviewer data. The AI co-scientist [4] structures generation–reflection–ranking–evolution agents into a self-improving tournament over hypotheses, with reported expert-validated outcomes in drug repurposing and antimicrobial resistance. Language agents for literature synthesis reach or exceed expert performance on retrieval-grounded tasks [10]. In the laboratory, Coscientist executed chemistry protocols from natural-language goals [8] and A-Lab ran autonomous materials synthesis campaigns [9]. Across all of these systems, quality control is architecturally *internal*. The reviewer agents, reflection stages, or tournament judges are chosen and hosted by the same operator, run inside the same platform, and are frequently instantiated from the same model family as the generators they judge. None of them commits to vendor heterogeneity in the reviewing layer.

The pattern persists through the 2025 generation of research agents: Curie hardens agentic experimentation with internal rigour modules [13], and Agent Laboratory attaches critic agents to its own pipeline [14]. Commercial platforms now advertise the transparency half of our proposal: Kosmos promises reports in which "every conclusion can be traced" to code or literature via its platform [15]. Traceability of that kind is audit by the generating operator on its own infrastructure. It is valuable, but it involves no second-party auditor and no off-platform, replayable ledger. This paper is about that distinction.

Why internal critique is not enough. Panickssery et al. demonstrate that LLM evaluators recognise their own generations and rate them preferentially; their fine-tuning experiments indicate that the capacity for self-recognition is what causes the inflation [5]. Zheng et al. catalogue further judge pathologies, including position, verbosity, and self-enhancement biases, even in strong models [6]. A recent large-scale evaluation sharpens why these matter for supervision rather than for benchmarking: across twenty-one judge models, high test-retest reliability coexists with severe position bias, and raw agreement overstates judge quality substantially once corrected for chance [7]. A judge can therefore be thoroughly reproducible and still be wrong in a fixed direction, which is precisely the failure a supervision loop must not inherit. These results concern single-model self-evaluation, but we expect the mechanism to generalise. Models that share a training pipeline are likely to share stylistic priors and knowledge gaps, and this places a same-vendor critic at the wrong end of the bias distribution just when its judgement matters most. We call this the *same-source supervision problem*. The remedy of drawing judges from different model families has precedent. Verga et al. score generations with a panel of evaluators drawn from three model families, precisely to dilute intra-model bias [16], and the same intuition has begun to appear in developer tooling as cross-vendor code review.[1] CrossAudit's claim is therefore not the heterogeneity idea itself but its packaging as a supervision protocol for research: versioned rules, a replayable ledger, deterministic precedence, and bounded escalation, run against a live pipeline. Cross-vendor pairing does not eliminate correlated error. Frontier models plausibly train on largely overlapping public corpora, so where the shared literature is wrong, both auditors can be confidently wrong together. What pairing does foreclose is the demonstrated configuration of the best-documented bias in this setting, namely self-preference, and it is intended to decorrelate vendor-idiosyncratic failure modes. What cross-vendor pairing cannot fix, a model-free layer must (§3, **I4**).

Supervision as an artefact. Software engineering solved an analogous trust problem not by making programmers infallible but by making the quality mechanisms *inspectable*: version control, continuous integration, review trails. Reproducibility initiatives in science point in the same direction; the unit of trust is the auditable record rather than the author's competence [11]. Deterministic screening of the research record also has its own lineage. statcheck recomputes reported statistics across the psychology

[1] E.g. MindStudio, "Cross-Vendor AI Agent Review: Why Claude Should Review Codex's Code and Vice Versa" (June 2026), `https://www.mindstudio.ai/blog/cross-vendor-ai-agent-review-claude-codex`, as accessed 30 July 2026, a workflow recommendation, without versioned rules, ledger, deterministic precedence, or escalation semantics.

Table 1: Supervision properties of related systems, as published. • = held by design; ∼ = partial or informal; – = absent. PoLL is an evaluation technique rather than a supervision loop; statcheck and the Problematic Paper Screener are deterministic by construction but run post-publication, where CrossAudit moves checks of that character before a claim leaves the repository. The CrossAudit column describes the protocol specification; §4 states which invariants the public reference implementation enforces versus targets.

	Critic from a different vendor	Versioned, decidable rules	Replayable public ledger	Deterministic precedence	Bounded loop, escalation
The AI Scientist [1]	–	–	∼	–	∼
AI co-scientist [4]	–	–	–	–	∼
PoLL judge panels [16]	•	–	–	–	–
Cross-vendor review advice (§2)	•	–	∼	–	–
statcheck, PPS [17, 18]	–	∼	∼	•	–
CrossAudit (this paper)	•	•	•	•	•

literature [17], and the Problematic Paper Screener runs mechanical detectors over millions of published papers [18]. Both are post-publication analogues of our deterministic check layer, which moves checks of that character upstream, before a claim ever leaves the repository. At the ledger end, blockchain-based research-integrity infrastructures pursue tamper-evidence with far heavier machinery;[2] Our position is that git, which is ubiquitous, free, and already attached to CI, permissions, and issue tracking, is ledger enough for the supervision trail. CrossAudit transplants this inspectability stance to agentic science: the supervision history itself becomes a first-class, versioned, citable research artefact. Table 1 compresses the comparison.

3 The CrossAudit protocol

3.1 Objects and roles

CrossAudit supervises a pipeline in which a **Generator** agent (vendor A) produces discrete *experiment increments*, commits under a designated path of a **Science Repository**. Each increment is audited by an **Auditor** agent (vendor $B \neq A$) against a versioned, human-authored rulebook, the **Constitution**, held with all *audit reports* in a separate **Audit Repository**. A **Deterministic Check Layer** (DCL), plain scripts with no learned component: schema validation, unit and dimension analysis, convergence assertions, provenance completeness, runs before any model reads anything. A **Human Principal** authors the Constitution and receives escalations, but sits outside the per-increment loop. Compute (in the reference deployment, jobs on cloud HPC) is decoupled from supervision: verdicts gate the *next* increment, never running jobs. Figure 1 traces one increment through the loop.

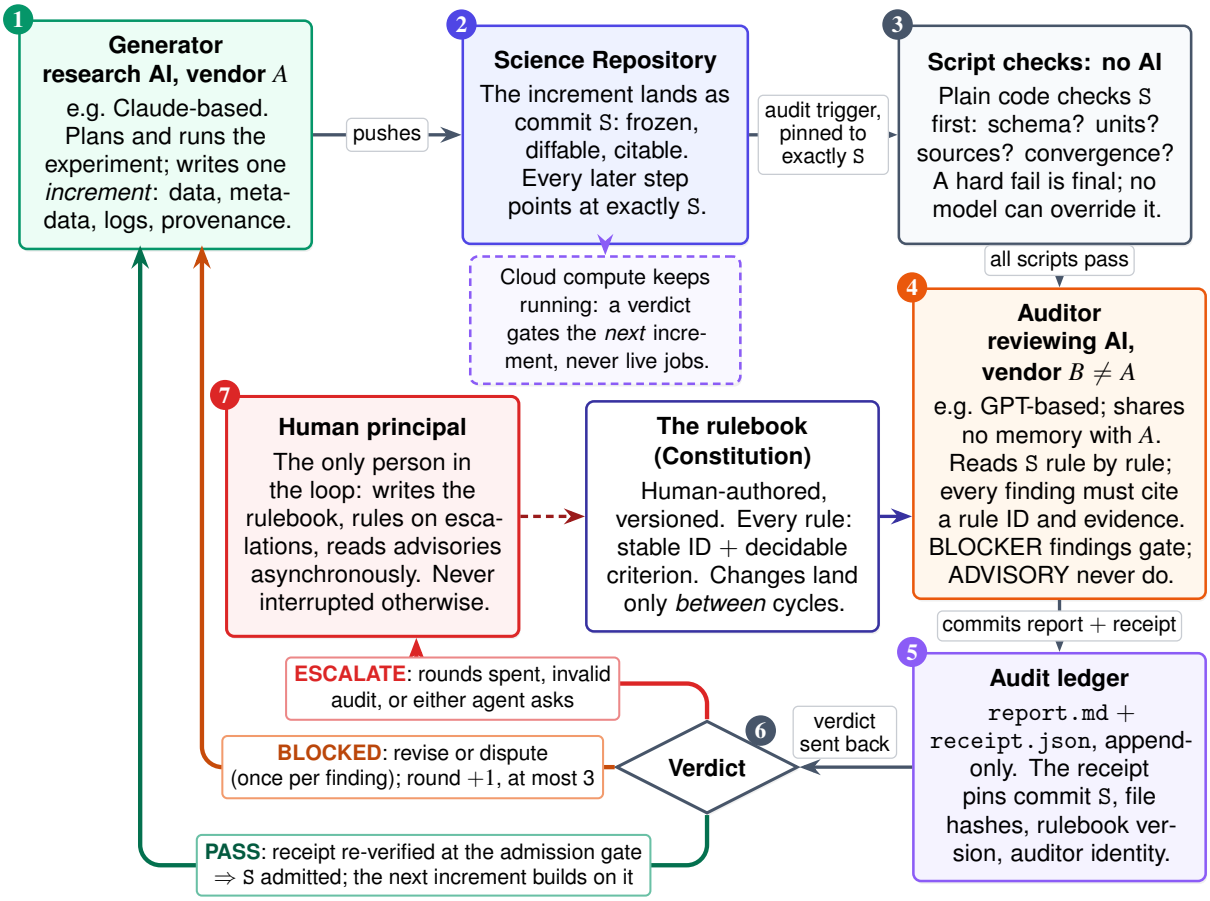
The two-repository split is separation of powers made mechanical: the Auditor holds no write access to the Science Repository, the Generator none to the Audit Repository, so neither agent holds credentials to alter the other's record; guarding each record against its *own* author additionally requires branch protection.

3.2 Eight invariants

An implementation is CrossAudit when it preserves the following. (**I7** and **I8** are revision-1 invariants, adopted from the cross-vendor audit of this repository described in §4.4.)

I1 — Heterogeneity. The Auditor's model family differs from the Generator's. This prevents the auditor from ever judging its own generations, the configuration in which self-preference bias has been demonstrated [5]; similarity-driven preference is thereby reduced, not eliminated. It is likewise designed to decorrelate vendor-idiosyncratic failure modes, an effect this paper does not measure (§4). Heterogeneity is, moreover, one of three isolations the protocol stacks: *parametric* (different

[2] E.g. the bloxberg consortium, `https://bloxberg.org`.



Every element on this page is a git commit or a GitHub issue: the entire supervision history is public and replayable by anyone.

Figure 1: One experiment increment through CrossAudit, read clockwise from the top left. The Generator writes an increment, which is frozen as a commit; scripts with no learned component check it first, and their word is final in both directions (**I4**); an auditor from a different vendor (**I1**) then reviews the same pinned commit against the human-authored, version-pinned rulebook, citing a rule for every finding (**I3**), and commits its report and receipt to the audit ledger (**I7**). The two repositories are mutually unwritable: the Auditor cannot touch the science record, the Generator cannot touch the rules or the reports. Three fates follow: admitted, only on a verified receipt (**I8**); sent back, at most three rounds with each finding disputable once (**I5**); or escalated to the human, who writes the rules and is interrupted only by these cases (**I6**); rule changes take effect between cycles, never mid-round, so the Generator never faces a moving target. Compute is decoupled: a verdict gates the next increment, never running jobs, and silent stalls are caught by a periodic dead-letter sweep. Every artefact is a commit or an issue (**I2**), so a third party can replay the whole history.

weights), *contextual* (no shared memory, the Auditor never sees the Generator's prompts, reasoning, or conversational state, only committed artefacts, closing the anchoring and sycophancy channels through which shared context sways a judge [19]), and *permissive* (no shared credentials; §3.1).

I2 — Ledger completeness. Every protocol artefact (increment, report, verdict, dispute, escalation, and every change to the Constitution) is a commit (or an issue) in one of the two repositories. No supervision state lives only in model context or platform logs. The history is replayable by a third party from the repositories alone, replay of the *record*; the model calls behind it are stochastic and not re-executable, and the reference ledger currently preserves parsed reports rather than raw exchanges.

I3 — Citation validity. An audit report must cite the rule IDs it applied and the commit hash of the Constitution version in force. A report that cites nothing, or an unverifiable version, is *invalid* and treated as an Auditor failure, it escalates; it can never pass an increment.

I4 — Determinism first. The DCL runs before any LLM audit. A DCL hard failure yields `BLOCKED` regardless of any model's opinion, and no model may waive a DCL verdict. Rationale: heterogeneous models still train on largely overlapping public corpora and hence can share their errors; the supervision layer whose failure modes are least entangled with any model is one with no learned component at all.

I5 — Bounded revision. At most `max_rounds` Generator–Auditor exchanges per increment (default three). Exhaustion escalates. This is the oscillation guard: without it, two mutually-correcting agents can disagree indefinitely, each round locally plausible and the pair jointly divergent.

I6 — Graded interruption. Findings carry exactly two severities. `BLOCKER` (objective defect: DCL failure, internal contradiction, missing provenance, method–declaration mismatch) gates the increment. `ADVISORY` (judgement: parameter taste, style, scope) is recorded for asynchronous human reading and never gates. Humans are interrupted only by `ESCALATE`. Running compute is never interrupted at all.

I7 — Receipt binding. Every verdict is a *receipt* bound to the audited commit SHA; to an artefact manifest (paths and content hashes) that the Audit Repository derives from that SHA itself, never from caller-supplied payload; to the Constitution, check-layer, and prompt versions applied; to the auditor's declared identity; and to the ledger commit of the report. A verdict that binds less is not a receipt.

I8 — Fail-closed admission. Protected actions (admitting the next increment, production submission, claim publication) proceed only on verification of a valid, current, matching receipt. Absent, stale, conflicting, unbound, or model-audit-less receipts deny by default and escalate: no audit, no admission.

Two severities only; in our judgement every additional level invites severity-inflation negotiation between the agents; where a finer distinction is needed it belongs in the rule's acceptance criterion, not in the escalation lattice.

Table 2 condenses the eight invariants into a reference card: the guarantee each one makes, the mechanism that enforces it, and how far the reference implementation carries that enforcement today. The statuses are the ones §4 accounts for; the distance that remains is enumerated in `ROADMAP-R2.md`.

3.3 The Constitution

The Constitution is the protocol's quality ceiling: the Auditor's blocking power may not, by rule, exceed the rules it is given (miscitation is caught by the dispute channel, not prevented), and the Generator is free to exploit whatever the rules fail to say. CrossAudit therefore treats it as law rather than documentation. Every rule carries a stable identifier (`CA-DATA-001`), a severity, a path scope, a *decidable* acceptance criterion, and a rationale. Decidability is the load-bearing requirement: a criterion a competent reader can

Table 2: The eight invariants as a reference card. "Enforced" means the reference implementation refuses the violating action in code; statuses follow the audited account in §4, and the implementation as a whole targets **I1**–**I8** while implementing a subset.

	Guarantee	Enforced by	Implementation status
I1	The Auditor never judges its own model family's output	Vendor pairing asserted from `crossaudit.yml` at dispatch	Enforced from configuration; vendor identity self-declared (§5)
I2	Supervision history replayable from the two repositories alone	Every artefact a commit or issue; append-only full-SHA cycle directories	Parsed reports preserved; raw model exchanges not yet captured
I3	A report citing no rules, or an unverifiable Constitution, can never pass	Reply validator converts malformed or citation-free replies to `ESCALATE`	Enforced
I4	No model may waive a scripted hard failure	Verdict synthesis is code; a DCL hard failure dominates any model verdict	Enforced
I5	Generator–Auditor oscillation terminates	Controller-managed rounds; exhaustion escalates	Enforced by the controller state machine
I6	Humans are interrupted only by escalation	Two-severity schema; `ADVISORY` never gates	Enforced in schema and router
I7	A verdict binds commit, manifest, and versions, or it is not a receipt	Receipt derived from the checked-out tree; verifier checks every bound field	Receipts emitted and verified; raw-exchange capture short of **I2**'s full intent
I8	No audit, no admission	Single-use, freshness-guarded admission; `DCL_ONLY` never admits	Verifier and controller enforce; binding admission to branch protection is a deployer toggle

evaluate against named artefacts without further judgement. What cannot be phrased decidably is either demoted to `ADVISORY` or (better) mechanised and moved into the DCL. Meta-rules govern the audit itself: missing evidence is a finding, never an implicit pass (`CA-META-001`); a report without citations is invalid (`CA-META-002`, enforcing **I3**); the Auditor may not issue a `BLOCKER` without citing a `BLOCKER`-severity rule, it does not legislate taste (`CA-META-003`); and matters beyond the increment-plus-Constitution evidence base must escalate rather than be guessed (`CA-META-004`). Constitution changes are commits reviewed by the human principal; the loop can propose rules but never enact them. (The name deliberately echoes, but must not be confused with, Constitutional AI [20]: there, a constitution steers a model's training-time self-critique; here, it is a runtime rulebook applied by a structurally independent auditor and versioned in the ledger.)

3.4 Rounds, disputes, termination

Round k for an increment: the Auditor publishes report R_k with a verdict. On `BLOCKED`, the Generator either *revises* (a new commit, opening round $k+1$) or *disputes* specific findings by rule ID, with grounds. A disputed finding returns to the Auditor once: it is withdrawn with a logged reason, or upheld. An upheld dispute at the round bound escalates. Disputes exist because the Auditor is fallible in both directions, a false-positive `BLOCKER`, silently obeyed, degrades the science as surely as a false negative, and the one-shot rule prevents the dispute channel from becoming the oscillation it exists to control. Two asymmetries make the channel safe, and the product line (§4.3) enforces both in code rather than by convention. The disputer states grounds and the Auditor rules, so nothing in a dispute can overturn a finding by itself; it buys a second reading, and the second reading is the Auditor's. And the Generator may not dispute at all. Grounds come from the principal, because a generator arguing with its own audit is exactly the anchoring channel **I1**'s contextual isolation exists to close. Contesting the *rule* rather than the finding is an amendment, which is dated and effective only between cycles. Termination is structural: each increment ends in `PASS` or in human hands after at most `max_rounds` exchanges. Figure 2 draws the walk.

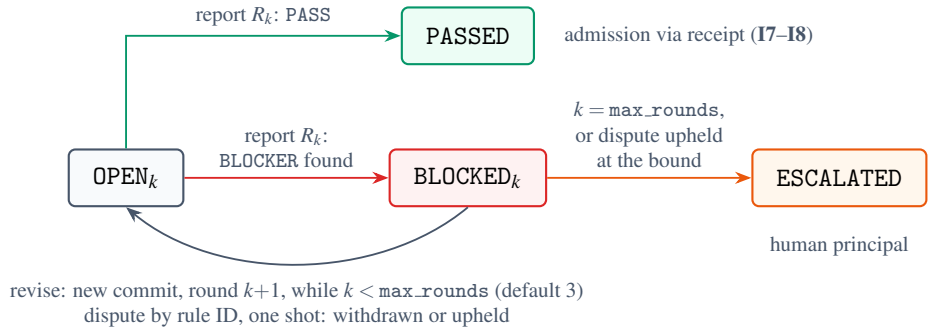


Figure 2: Termination as a walk on four states. Every path from `OPEN` ends in `PASSED` (admission gated by receipt) or `ESCALATED` (a human decides); the only cycle in the graph is the revision loop, and the round bound cuts it after `max_rounds` exchanges. The dispute edge re-enters the same loop rather than adding one: a finding may return to the Auditor once.

4 Reference implementation and deployment

4.1 Mechanics

The reference implementation[3] uses no infrastructure beyond GitHub itself and two model API endpoints. A push under `experiments/` triggers a cross-repository dispatch carrying the commit SHA; the audit side derives the audited scope from the SHA itself and the round from its own report index, caller-supplied payload is treated as untrusted hint. The Audit Repository's workflow checks out the science repository *pinned at the audited SHA*, asserts **I1** from configuration, runs the DCL, then invokes the Auditor via an OpenAI-compatible endpoint with the Constitution, the DCL output, and the increment content as fenced data. The report is committed to the ledger; the verdict is dispatched back; the science repository routes it, proceed, revise, or open an escalation issue. The whole implementation is three workflow files and two Python programs (a check runner and an auditor runner) of a few hundred dependency-light lines; configuration is a single `crossaudit.yml` declaring vendors, `max_rounds`, and escalation policy.

Status against the invariants. Per-invariant enforcement and status are Table 2's job (§3.2); what belongs here is how those statuses were earned, because the route is itself evidence for the protocol. Following two rounds of cross-vendor audit of this repository (§4.4; both reports and dispositions committed under `audits/`), scope became SHA-derived, rounds ledger-derived rather than generator-controlled, receipts full-field, and the reply validator strict; truncated, unreadable, or symlinked inputs now escalate rather than pass, and keyless runs yield `DCL_ONLY`, which the router treats as non-admission. A third cross-vendor read (also committed) then found the sharper failure mode: the controller state machine and full receipt verifier existed but were not wired into the execution path, dry-run admission, a hard-coded report path that broke on disputed rounds, a constitution check that could never fire. Those wiring defects are fixed; what remains unimplemented is the deployer-side binding of the admission status to branch protection, and raw model-exchange capture in the audit runner. The public implementation should therefore be read as *targeting* **I1–I8** and implementing selected checks; the remaining distance is enumerated in `ROADMAP-R2.md`.

Three details with protocol weight. First, the auditor runner validates the model's reply against a fixed schema (verdict, findings with severities and rule citations) and any malformed or citation-free reply is converted to `ESCALATE` (**I3**); a familiar failure of LLM judges (the confident empty report) is thereby converted from silent passage into an escalation (when the DCL has also hard-failed, the verdict remains `BLOCKED` and the auditor failure is recorded as an `audit_integrity` field in the receipt; router-side escalation on that field is revision-2 work). Second, increment content is passed to the Auditor as

[3] `https://github.com/dongzhaohe321418-lab/crossaudit` (MIT licence).

fenced, size-bounded content with the system prompt instructing that embedded instructions be ignored, a best-effort defence whose residual risk §5 states plainly; the reference Auditor has no tools. Third, verdict synthesis is code, not model output: DCL hard failures dominate any model verdict (**I4**).

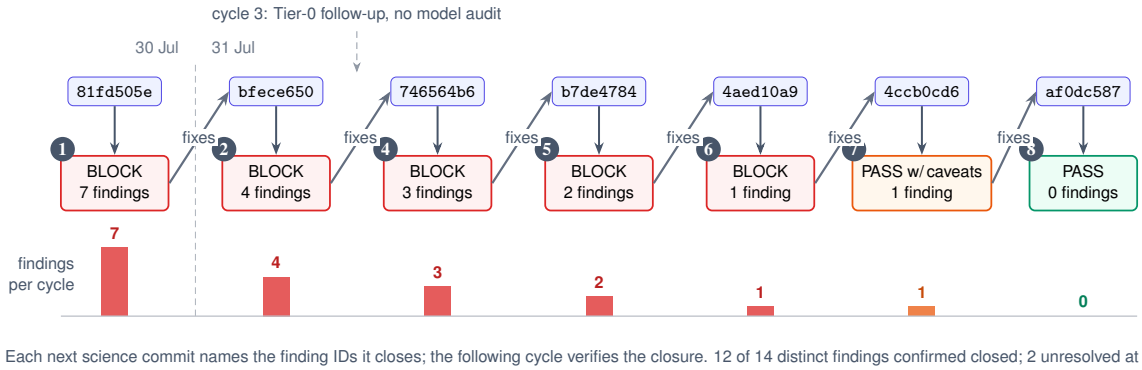
4.2 A live deployment

A closely related variant of CrossAudit runs live in the first author's computational-chemistry pipeline (perovskite additive screening). The generation side is a Claude-based research agent that plans calculations, launches them on Alibaba Cloud E-HPC and a GPU worker, and pushes every increment to a private science repository whose audit policy, an epistemic-status ledger separating established, bounded, and provisional claims, together with a standing retraction log, is itself a committed, versioned document. A controller opens each cycle against a fixed science commit. The audit request pins a SHA-256 manifest of every tracked file. A Codex-CLI auditor (OpenAI lineage, so **I1** holds against the Claude-based generator) writes four artefacts into an append-only per-cycle directory in the audit repository: a human-readable report, a machine-readable result, run metadata, and a per-file hash manifest. All sit on one long-lived `audit` branch that later cycles may not rewrite. The controller enforces this by validating every audit commit before recording a cycle as final, which detects a rewrite rather than preventing one; prevention would need branch protection. The controller's own orchestration state deliberately lives outside both repositories, so tampering by either agent with the record of its own supervision is detectable against controller-held state.

Three deliberate divergences from the reference design should be named plainly: the deployed auditor is a tool-bearing CLI agent rather than the reference's toolless one, contained instead by that commit validation and the append-only cycle layout; the externalised controller state means the deployment's history is fully replayable from the repositories *plus* that state, a weaker property than **I2**'s repositories-alone ideal; and findings carry a four-level severity ladder feeding scope-level gates, rather than the protocol's two levels, a live experiment running against §3.2's two-level argument, whose outcome the telemetry will eventually adjudicate.

Every input and output lives on GitHub; the system is designed so that any single component (generator, auditor, controller host, workstation) can fail without loss of ledger state.

Two refinements of the deployed system go beyond the reference implementation and are worth adopting upstream. First, findings are *evidence-typed*: each carries a policy citation (section and rule ID), a deterministic check identifier where applicable, and a copy-paste *reproduce* command, so a human can re-derive any deterministic finding in seconds. Second, blocking is *scope-granular*: a finding blocks named scopes, production-job submission, claim publication, rather than the pipeline wholesale, refining **I6**'s graded-interruption principle. Human authority enters through escalation, plus one deployment-specific addition: a standing principal-approval gate on production launches, recorded in the ledger. The loop is live in the plainest sense: its first cycle (opened and closed within six minutes) returned a `BLOCK` decision on deterministic grounds (manifest-hash disagreements and commit-provenance defects), freezing production submission and claim publication until resolved. Audits gate the record and future submissions, never running jobs: E-HPC work in flight is never interrupted. A redacted public mirror of the deployment's finalised cycles is maintained in the reference repository (`ledger-mirror/`; scientific magnitudes masked, structure, rule citations and reproduce-command shapes verbatim). Since that first cycle the ledger has accumulated seven model-audited cycles across two days (Figure 3): decisions ran `BLOCK` five times, then `PASS WITH CAVEATS`, then `PASS`, while findings per cycle fell 7, 4, 3, 2, 1, 1, 0. Of fourteen distinct findings, twelve are behaviourally confirmed real: each revise leg is a science-repository commit naming the finding IDs it closes, and the following cycle verifies the closure; two remain unresolved at the data freeze. Absent the loop, all twelve confirmed defects would have proceeded downstream unchallenged. One leg (the ledger's numbering gap at cycle 3) ran as a deterministic-tier follow-up without a model audit and is reported as-is. The generator's session state was never committed, so no in-context self-review arm can be reconstructed retrospectively; that absence is itself evidence for **I2**. The mined, structurally redacted cycle records are frozen in this repository (`experiment/v3/real-ledger/`).



Each next science commit names the finding IDs it closes; the following cycle verifies the closure. 12 of 14 distinct findings confirmed closed; 2 unresolved at freeze.

Figure 3: The live deployment's first convergence arc, mined from its audit ledger at the data freeze (science `af0dc587`, audit `88b92429`; structurally redacted records in `experiment/v3/real-ledger/`). Top lane: the audited science commits, with the day boundary from the ledger's timestamps dashed. Middle lane: the audit cycles, coloured by decision, with the deployment's own numbering gap preserved (the dashed pointer marks leg 3, a deterministic-tier follow-up without a model audit). Bottom: findings per cycle on a shared baseline, falling 7, 4, 3, 2, 1, 1 to 0 as decisions move from `BLOCK` to `PASS`; absent the loop, the twelve behaviourally confirmed defects would have proceeded downstream unchallenged.

The loop has since kept running. By 3 August the ledger held 21 model-audited cycles carrying 23 distinct findings, 20 of them behaviourally confirmed. After the first arc converged, two further increments passed on their first audit; the sustained development that followed settled into a working rhythm of one to three findings per cycle, with a median closure lag of one cycle and, in this deployment, a turnaround typically inside the hour. The findings themselves were graded on the deployment's own four-level ladder as three `CRITICAL`, thirteen `HIGH`, four `MEDIUM`, and three `LOW`; twelve of the twenty-three cite a single provenance gate, which is the kind of concentration that tells a principal where the next deterministic check should go. The scope-granular gate was doing its intended work throughout: at this snapshot a claim-publication action stood withheld, pending a clean verdict on the increment beneath it. Two caveats on reading these numbers. The auditor model was not held fixed across the run (four identifiers appear in the metadata), so the series mixes configurations and cannot support arm comparison. And the v3 study's Part B labels remain frozen at the 31 July snapshot; the later cycles are reported here as operational telemetry only.

Costs for the reference design are modest. The DCL is free. A single fenced-increment audit call scales with the size of the increment, not of the computation behind it, estimated at cents for a few hundred lines of results and metadata, while the experiment it certifies may represent hours of HPC time. (The deployed variant's tool-bearing auditor reads more and costs more; per-cycle cost reporting is deferred with the rest of the telemetry.) Auditing costs roughly what it costs to read the documentation, not what the compute behind it cost. Two caveats: multi-round exchanges multiply audit calls, and escalations spend human time, which is the truly scarce resource here.

The deployment's role is existence proof for the deployed variant: the loop has run unattended, at single-researcher scale, on commodity infrastructure, and has completed one full convergence arc from `BLOCK` to `PASS`. Its first catch was attributable to the deterministic layer; the fourteen findings that followed came through the tool-bearing auditor, and what fraction of them a script could have caught (the mechanisability of the confirmed set) is a registered endpoint of the follow-up study, adjudication pending, rather than a claim made here.

4.3 From a reference implementation to a product line

The reference implementation above is deliberately minimal, and its minimality has a cost: to use it, a researcher must first absorb a vocabulary (increments, Constitution, receipts, cycles), hand-write a rulebook, and learn a command surface. That is the right shape for a protocol study and the wrong shape for adoption. A separate line of work therefore rebuilt the loop as an installable package, with the engine's

invariant semantics carried over unchanged.[4]

Its product judgement was to delete the act of writing rules and leave only stating requirements and assenting to them: the rulebook stops being the user's obligation and becomes the system's output, distilled from a conversation, shown for assent, and committed as a version-pinned Constitution. Three things that emerged from that work are protocol content rather than packaging, and we record them here. None of this is a second evaluation. The product line is not measured anywhere in this paper, and the trial of §4.4 was run against the reference implementation's scorer alone; what a second implementation provides is the ordinary benefit of writing a specification down twice, which is that the places where the first writing was underspecified become visible.

Admission is a tier, and a deployment should be told which one it holds. **I8** says that protected actions proceed only on a verified receipt, and §3.2 notes that binding the check to branch protection is a deployer action. That framing is too binary to be useful in the field, because deployments sit at different distances from the guarantee. The product line names five positions and reports the one a deployment can actually claim: *local*, where the history is the operator's to rewrite and can hold no one to account; *remote*, where history escapes unilateral control; *paired*, which adds privilege separation between the agents; *enforced*, where a failed audit refuses the merge; and, between the last two, *verified-notification*, where the verdict is published and checkable but nothing is refused. That fifth name is the one worth having. A paired deployment whose gate does not exist publishes verdicts, and letting it be called *paired* invites the reading *gated*; announcing a verdict while implying refusal is worse than saying nothing, because it manufactures confidence the ledger cannot support. The evidence rule matters as much as the ladder: absent evidence never counts in favour. The controller's persistence and atomicity are established by taking the lock rather than by reading a configuration field, the branch rules are read from the platform rather than from the deployment plan that was supposed to set them, and a deployment that has not probed its own gate is told that a gate nobody has looked at is not a gate.

A component that decides what the auditor sees must itself be audited. Putting the loop behind a single conversational surface introduces a component the reference design does not have: a router that decides whether an utterance is a change to the work, a change to the rules, a dispute, or a query. This is a real addition to the trusted base. A router that could quietly decide what the auditor never sees would be a third agent, unaudited, holding precisely the power the protocol exists to distribute. The resolution follows from **I2** once the invariant is read as covering discretion and not only artefacts: each routing decision is a ledger entry carrying the utterance, the lane chosen, the confidence, and the action actually taken, so a misroute is visible and reversible rather than silent. Below a confidence threshold the system asks instead of guessing, which is the fail-closed default in conversational form. The same reading forbids a second, subtler leak: the router forwards the principal's words, never the Generator's, so the Generator's prompts, reasoning, and self-justification stay on their own side of the boundary.

Nothing in the eight invariants mentions science, and an implementation can say so honestly. The protocol is domain-general by construction, but the reference deterministic layer is not: its four checks assume experimental data. Generalising meant replacing them with a domain-neutral default (a file claiming to be JSON or YAML must parse, a declared input must exist within the audited scope, an internal link must resolve, and placeholders must not survive into audited work) and distributing domain packs as plugins. Two disciplines came with that. Packs are loaded only when named in an allowlist rather than discovered, because a plugin entry point is arbitrary code execution in a process that is about to hold credentials and write the ledger. And where no pack exists for a field, the deterministic layer contributes only those four checks while the model audit carries the rest, so **I4**'s mechanical guarantee thins; an implementation in that position should say so rather than let a uniform interface imply uniform

[4] `https://github.com/dongzhaohe321418-lab/crossaudit_v3` (MIT licence), at version 3.2.0: around ten thousand lines of dependency-light Python with four thousand lines of tests. The research repository does not chase the product; the protocol's authoritative text remains this paper.

strength. The same reticence governs what a supervision interface may display: a figure the ledger cannot support is rendered as not measured rather than as zero, since "no escalations" and "we never looked" are different claims and only one of them deserves a large font.

4.4 An exploratory seeded-defect trial, audited and corrected

We attempted a pre-registered, sealed seeded-defect pilot. The attempt did not survive contact with our own protocol. A cross-vendor audit of this repository, in which a second-vendor agent reviewed the generating agent's work, returned a BLOCK-grade report whose findings voided the "sealed" framing and several companion claims; the report and our dispositions are committed at `audits/`. We adopted its findings in full. What follows are the corrected, exploratory results, which proved more instructive than the experiment we had intended to run.

Design and execution. A deterministic generator produced 30 synthetic increments in the reference format: 10 clean and 20 defective, carrying 43 seeded defects across ten registered classes, four of them mechanisable and six requiring reading. A SHA-256 digest of the defect key was committed at registration. Three arms then ran over identical inputs with the identical Constitution. The first was the deterministic check layer. The second was a same-family arm: fresh-context Claude subagents, one per increment, with no shared memory (session sampling, a deviation we disclose). The third was a cross-vendor arm: `gpt-5.1` at temperature 0, run by the frozen runner in CI.

What the audit found. We checked each finding and accepted all of them. The key file was public from the registration commit onward. This was our mistake, a mis-pathed ignore rule, and the key is in any case derivable from the public seeded generator, so no blinding claim survives; what remains is procedural blinding of fresh-context auditors, which the ledger cannot verify. The headline same-family score depended on a scoring-map expansion made after the outputs were committed. Under the frozen map the arm scores 38/43; under the adjudicated map, with corrections logged, it scores 43/43. The former is the reportable score and the latter is exploratory. The deterministic layer's five L6 catches contradict the registered taxonomy: the class was misregistered as model-only when its flag component is mechanisable. The trial is component-level throughout. The corpus was read directly, with no dispatch pipeline and no DCL precedence, so it is not an end-to-end run of the reference implementation. Finally, the trial's own outputs violate **I3**: the runner recorded no Constitution hash, a defect one of the auditors itself flagged (below).

Arm	Recall lenient / strict	Chance floor lenient / strict	Blockers on 10 clean	Verdicts (of 30)
Deterministic layer	17/43 [a]	4.8 / 4.2	0	25
Claude-family (subagents)	38 / 38 [b]	22.4 / 16.3	0	30
GPT-family (`gpt-5.1`, frozen runner)	41 / 37 [c]	31.8 / 24.4	26	20

[a] 12/12 of its registered scope + 5/5 cross-hits on misregistered L6. [b] 43/43 under post-hoc adjudicated scoring. [c] all ten clean increments blocked (10/10 false-block verdicts); many findings letter-valid on inspection; per-finding adjudication pending. Strict-tier scores assemble rule and location evidence across findings rather than binding one finding per defect, and so overstate defect-level localisation. Chance floors: 2000-shuffle permutation null, seed 20260731; `experiment/results/NULLCHECK.json`.

The vendor split is the result. The cross-vendor arm scored 41/43 at the lenient tier but 37/43 at the strict tier. It emitted 113 findings to the same-family arm's 63, and it returned BLOCKED on *all thirty* increments. That last figure deserves weight, because a stub auditor that blocks unconditionally would reproduce it exactly, including the 20/30 verdict accuracy. It is the reliability-without-validity failure in its purest form [7]: a perfectly consistent judge carrying no information about the increment in front of it. Raw recall cannot settle the comparison, because it rises with the number of findings an arm emits; this is the same reason raw agreement overstates judge quality in the benchmarking literature [7]. A

permutation test (2000 shuffles of the increment-to-defect map, holding each arm's outputs fixed so that volume and citation habits are preserved; seed 20260731, `experiment/score_nullcheck.py`) puts the random floors at 4.8, 22.4, and 31.8 of 43 for the three arms. Once corrected, the headline lenient comparison is within noise, and the strict-tier order reverses: chance-corrected agreement is 0.81 for the same-family arm against 0.68 for the cross-vendor arm. Inspection of the clean-increment blockers still suggests that many are letter-valid readings of an ambiguous rule, but per-finding adjudication was not logged, and the claim as originally framed was unfalsifiable: any disagreement would have counted as support. The rule at issue, `CA-DATA-001`, requires "each numeric entry" in the results file to carry a unit and a source; the corpus's convergence blocks carry units but no sources. The same-family auditor, like the check scripts and like the corpus author, read the rule by its intent and raised nothing. The foreign auditor enforced its letter. What survives is narrower but real. On identical inputs, the two vendors read the same rulebook differently, which is **I1**'s premise. The trial nevertheless measures disagreement rather than discrimination, and a 10/10 false-block rate on clean increments means that in this configuration every increment escalates, so the $O(\text{escalations})$ economics of §6 hold only under a far better-calibrated auditor or a sharper Constitution. The practical outcome was a Constitution amendment: scope the rule decidably.

What we can still claim. The limits should be stated plainly. This is a component-level, checklist-aligned trial (the prompt names the defect families the corpus injects) on template-generated data, with the same-family arm's execution self-attested and n=43. Zero blockers on ten clean increments bounds that arm's false-positive rate only below ≈26% (one-sided 95%), and its advisory burden on the clean increments was five findings across five of ten. None of this is confirmatory evidence about detection rates. Three things the trial does establish. The model layers read: they caught 26 to 31 defects beyond the scripts' scope. Vendor heterogeneity produced observably decorrelated readings. And the repository's own disciplines (append-only outputs, logged adjudication, a committed cross-vendor BLOCK with itemised dispositions) were applied to the evaluation itself.

4.5 What a reader can check

Three artefacts are public and versioned, and they answer different questions. The reference implementation carries the protocol as specified here, together with the controller, the receipt verifier, and their regression tests. The evaluation directory carries the trial corpus, the frozen scorer, the permutation null (seed 20260731) and its output, so every number in §4.4 regenerates from a clean checkout. The audit directory carries the cross-vendor reports written against this repository and our itemised dispositions, including the ones that voided our own claims. The live deployment's supervision records are mined into the same repository in structurally redacted form: cycle identities, decisions, severities, rule citations, and finding lifecycles verbatim, scientific magnitudes masked.

5 Threat model and limitations

Below is what CrossAudit withstands and what it does not.

- **Self-preference and vendor-correlated error** are addressed by **I1**. *Corpus*-correlated error is not: where the shared literature is wrong or outdated, both vendors can agree confidently and wrongly. The mitigation is **I4** for everything mechanisable; for what is not, it is the human's asynchronous reading of `ADVISORY` trails. This residual cannot be assumed reducible by adding more LLM auditors, because all of them plausibly train on largely overlapping public corpora.
- **Prompt injection.** An increment could embed text addressing the Auditor. The Auditor runs toolless against a fixed reply schema, with increment content fenced as data; and because DCL verdicts are non-overridable, injection cannot waive the layer where hard defects are caught. Injection that shapes the *LLM-side* report remains partially open, as it does for LLM-judge systems

generally. This bullet describes the reference implementation; the deployed variant's tool-bearing auditor forgoes the toolless defence, a divergence §4 names together with its containment limits.

- **Auditor failure.** Empty, malformed, or rule-free reports convert to escalation (**I3**), never to silent passage. The cost is human attention, which is the correct currency for an unreliable auditor.
- **Oscillation and deadlock** are bounded by **I5**; the failure mode degrades to a human decision with the full exchange on the ledger, which is the intended floor.
- **Goodharting the Constitution.** A capable Generator will satisfy the letter of decidable rules. Decidability makes this visible rather than solved: the human reviews rule–outcome drift on the ledger and legislates accordingly. The Constitution is a living document, and its growth out of escalations is the system learning. A related boundary matters wherever a deployment lets the principal shape how the Generator works, through style guides, domain conventions, or worked examples. Such material may change how the work is done; it must never change where the Generator may write or who judges it. Guidance that could address the Auditor would be an unversioned rule, and guidance claiming wider write access than the path guard allows is only a text with an opinion.
- **Collusion** has no out-of-band channel in the reference design; coordination must ride the committed artefacts, where it is preserved for inspection. Encoded coordination *within* artefacts, and any channels a tool-bearing auditor acquires, remain possible. The realistic soft risk is convergent style across many rounds, and it is met by the Constitution's review cadence.
- **The operator.** CrossAudit supervises the agents, not the principal. Selective publication, pre-publication history rewriting, and audit re-rolling by the operator all sit outside the protocol's guarantees. External anchoring (signed pushes, transparency logs) would extend tamper-evidence to the operator; until then, "third-party-auditable" should be read as *internally consistent as published*. Relatedly, the DCL scripts and the Auditor's system prompt must live under the same ledgered, human-reviewed change control as the Constitution, because a Generator-editable check layer would hollow out **I4**.
- **Vendor identity is self-declared.** The ledger proves that an exchange occurred; it does not prove the identity of the models behind OpenAI-compatible endpoints. **I1** is therefore asserted from configuration rather than verified cryptographically. Attested inference or vendor-signed responses would close this gap; today it is an honest limit on what "third-party-auditable" can mean. (The reference example moreover shares one dispatch token across both workflows, so credential separation is likewise configuration, not proof.)
- **Confidentiality.** The protocol ships every increment, possibly unpublished and patentable material, to a second vendor's API. Data governance of that call (agreements, retention, disclosure risk) sits outside the protocol and must be settled per deployment.
- **Scope.** CrossAudit audits the research *record*, not the world. It cannot detect a miscalibrated instrument, a wrong dataset honestly described, or fraud upstream of the repository. It is also currently specified for single-generator pipelines; multi-agent generation raises attribution questions we do not address here.

6 Discussion

Continuous integration for scientific claims. The ambition is what CI achieved for software: not proof of correctness, but a cheap and inspectable floor under quality. Human judgement is then redeployed to where it discriminates (the rules, the escalations, and the science) rather than spent uniformly on every increment. We believe the analogy holds in one further respect. CI spread without institutional permission; a single developer could adopt it unilaterally, and its artefacts, the badges and the logs,

became social proof. One disanalogy must be owned: CI's checks are deterministic, while CrossAudit's distinctive layer is a stochastic judge. The DCL exists precisely to keep the gate's hard core deterministic. CrossAudit is designed for the same adoption path: one researcher, two repositories, an afternoon.

Relation to platform-scale systems. CrossAudit is complementary to, not competitive with, systems such as the AI co-scientist [4] or the AI Scientist [3]. Those systems generate; CrossAudit supervises whatever generates. Indeed, the sharpest version of our proposal is addressed to exactly such platforms. Their internal critique stages appear architecturally straightforward to re-instantiate across vendor lines and to externalise onto public ledgers, although the organisational and confidentiality costs are not ours to estimate. The resulting audit trails would give publishers and funders, who are currently asked to trust process descriptions [12], something they can actually inspect.

What the ledger buys science socially. A complete supervision history, public where the operator publishes it, changes the character of machine-generated results. A claim then arrives not as prose plus reputation but as prose accompanied by its audit record: which rules it was checked against, what was contested, and what a differently-trained model conceded or refused. Reviewers can spot-check the ledger instead of re-deriving trust from nothing.

The ledger as a data asset. A CrossAudit deployment produces, as a by-product, something the field currently lacks: a corpus of *supervised machine science*. Each tuple of increment, findings, dispute, and resolution is a labelled example of scientific work judged by an independent party under explicit rules, and therefore high-grade training signal for better generator and auditor agents alike. Aggregated across deployments, failed audits would form a public error taxonomy of agentic research: what machine-generated science actually gets wrong, per rule, per field, over time. Whoever runs the loop accumulates this asset. A community that runs it openly accumulates it as a commons.

Constitutions as community objects. Because the Constitution is a plain, forkable file with stable rule IDs, it can outgrow any single deployment. A research community can maintain a shared domain constitution in the way communities maintain style guides and lint rules. A journal could publish the constitution it expects machine-assisted submissions to have been audited under, and accept the ledger link alongside the manuscript, much as data-availability statements are accepted today. Funders could write "public supervision ledger" into their terms. For regulated settings, pharmaceutical and clinical computation among them, the ledger's attributability and its contemporaneous, hash-manifested record resemble properties that audit regimes already demand of human work, though we claim compliance with no specific regime. A repository badge (increments audited, escalations raised, constitution version) would then do for machine science what CI badges did for software: it would compress an inspectable process into a glanceable, verifiable signal. None of this requires new technical infrastructure. It requires only that the artefact exist, which a conforming deployment produces.

A ratchet for standards. The Constitution need not be static. Because rules are commits and receipts pin the version in force (**I7**), standards can tighten over time without ambiguity about which version governed which increment. The safe dynamics have three channels. First, shadow-mode promotion: a candidate stricter rule enters as `ADVISORY`, rehearses enforcement on every cycle without gating, and accumulates ledger evidence on hit rate, dispute rate, and would-be blocking cost; the human promotes it to `BLOCKER` only when the record supports it. Second, threshold ratchets: numeric criteria tighten stepwise as the generator's competence grows, each step a commit with its rationale. Third, agent-proposed amendment: either agent may draft a rule change from what the ledger shows it. The loop can propose; only the principal enacts. One rule matters most. Standards freeze within a cycle and move only between cycles, because an auditor that could raise the bar mid-round would face the generator with a moving target and dissolve the termination guarantee of **I5**.

From copied glue to an installable protocol. The reference implementation ships as copy-in glue: a deployer vendors `checks/` and `controller/` into an audit repository and edits three workflow files. The product line described in §4.3 replaces that with an installable package whose command surface is the loop itself: scaffold a project, run the deterministic layer and the auditor against a pinned commit, verify a receipt, admit. The argument for packaging is not only convenience. Vendored copies drift, so two deployments can silently run different verifiers, and a receipt that pins the Constitution, the check sources, and the prompt, but not the machinery that verified and admitted on their basis, is bound one layer short of **I7**'s intent. A release-versioned verifier closes that layer: the receipt can record the package version and distribution hash alongside the hashes it already carries, and the enforcement code becomes a pinned, citable dependency rather than an unversioned copy. This binding is specified but not yet emitted, and it is the next thing we would fix. Packaging also gives field adaptation a natural unit, since check packs ship as plugins and DCL coverage can circulate between groups in the way we argue Constitutions should. The cost is a new trust surface, the package supply chain, for which hash-pinned installs and signed releases are the standard mitigations.

A console for the human principal. The ledger is git-native on purpose, and it stays so. Raw git is nevertheless a demanding reading surface for the one human the protocol keeps in the loop, and **I6** makes that human's asynchronous reading load-bearing. The product line therefore carries a supervision console over the ledger: cycle timelines, the advisory backlog by rule and hit rate, an escalation inbox assembling what an adjudication actually needs, and the telemetry a standards ratchet would read. One design rule carries protocol weight: the console writes nothing of its own. Every action it offers runs the same verbs the command line offers, so each materialises as a commit or an issue through the authenticated paths the agents already use, the ledger stays complete (**I2**), and the console remains derivable from it. A supervision interface with a private database would accumulate exactly the unversioned, unreplayable state the protocol exists to exclude. Three refusals follow from the same principle and are worth stating because a supervision display that breaks them is worse than none: it does not show a figure the ledger cannot support, it does not colour a step green before it happened, and it does not imply that it acted on its own. There is a boundary case that deserves its own honesty. Work in flight is not yet in the ledger, so progress reporting is a view rather than a record: it lives in memory and dies with the process. The ledger holds every committed round but cannot know that a round was cut off, so an interrupted run must be marked as interrupted rather than left to read as finished.

Management by exception, at research scale. As agent labour becomes cheap, the binding constraint of agentic science shifts from compute to human attention. CrossAudit's economic content is a reduction in *gating* cost from $O(\text{increments})$ to $O(\text{escalations})$, conditional on escalation volume staying low. Our own trial shows the failure mode plainly: the cross-vendor arm's 10/10 false-block rate would make escalations $O(\text{increments})$. The economics therefore hold only with a calibrated auditor and a decidably scoped Constitution, and they must be measured rather than assumed. This is the managerial precondition for one researcher credibly directing several agent pipelines at once. In principle it should extend beyond science to any agentic work product that arrives as discrete increments whose quality is partially rule-expressible, from code to quantitative analysis, a conjecture this paper does not test.

7 Conclusion

CrossAudit reframes supervision of autonomous research from a platform feature into a public, versioned artefact produced by structurally independent reviewers. Its ingredients are plain: a second vendor, a rulebook with stable IDs, scripts that run before any model, a bounded loop, and git underneath. We chose them because a single researcher can adopt all of this without asking anyone's permission. The protocol's eight invariants are a deliberately small set. Together they buy structurally independent review, a re-inspectable history, and human authority at the boundary; everything else in this paper is reference

detail, and the reference repositories are public. An AI scientist should not grade its own homework. With two repositories and two API keys, it no longer has to.

Acknowledgements. The CrossAudit reference implementation and the deployment described in §4 were developed with the assistance of the very class of agentic tools this paper proposes to supervise.

References

[1] C. Lu, C. Lu, R. T. Lange, J. Foerster, J. Clune, and D. Ha. The AI Scientist: Towards fully automated open-ended scientific discovery. *arXiv:2408.06292*, 2024.

[2] Y. Yamada, R. T. Lange, C. Lu, S. Hu, C. Lu, J. Foerster, J. Clune, and D. Ha. The AI Scientist-v2: Workshop-level automated scientific discovery via agentic tree search. *arXiv:2504.08066*, 2025.

[3] C. Lu, C. Lu, R. T. Lange, Y. Yamada, S. Hu, J. Foerster, D. Ha, and J. Clune. Towards end-to-end automation of AI research. *Nature*, 651:914–919, 2026.

[4] J. Gottweis, W.-H. Weng, A. Daryin, T. Tu, et al. Towards an AI co-scientist (later retitled "Accelerating scientific discovery with Co-Scientist"). *arXiv:2502.18864*, 2025.

[5] A. Panickssery, S. R. Bowman, and S. Feng. LLM evaluators recognize and favor their own generations. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. arXiv:2404.13076.

[6] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, et al. Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[7] J. D. Norman, M. U. Rivera, and D. A. Hughes. Reliability without validity: a systematic, large-scale evaluation of LLM-as-a-judge models across agreement, consistency, and bias. *arXiv:2606.19544*, 2026. arXiv:2306.05685.

[8] D. A. Boiko, R. MacKnight, B. Kline, and G. Gomes. Autonomous chemical research with large language models. *Nature*, 624:570–578, 2023.

[9] N. J. Szymanski, B. Rendy, Y. Fei, R. E. Kumar, et al. An autonomous laboratory for the accelerated synthesis of inorganic materials. *Nature*, 624:86–91, 2023.

[10] M. D. Skarlinski, S. Cox, J. M. Laurent, J. D. Braza, et al. Language agents achieve superhuman synthesis of scientific knowledge. *arXiv:2409.13740*, 2024.

[11] M. Baker. 1,500 scientists lift the lid on reproducibility. *Nature*, 533:452–454, 2016.

[12] Editorial. AI scientists are changing research — institutions, funders and publishers must respond. *Nature*, 651:853–854, 2026.

[13] P. T. J. Kon, J. Liu, Q. Ding, et al. Curie: Toward rigorous and automated scientific experimentation with AI agents. *arXiv:2502.16069*, 2025.

[14] S. Schmidgall et al. Agent Laboratory: Using LLM agents as research assistants. *arXiv:2501.04227*, 2025.

[15] Edison Scientific. Kosmos: An AI scientist for autonomous discovery. Announcement, November 2025. `https://edisonscientific.com/news/announcing-kosmos`.

[16] P. Verga, S. Hofstätter, S. Althammer, et al. Replacing judges with juries: Evaluating LLM generations with a panel of diverse models. *arXiv:2404.18796*, 2024.

[17] M. B. Nuijten, C. H. J. Hartgerink, M. A. L. M. van Assen, S. Epskamp, and J. M. Wicherts. The prevalence of statistical reporting errors in psychology (1985–2013). *Behavior Research Methods*, 48:1205–1226, 2016.

[18] G. Cabanac, C. Labbé, and A. Magazinov. The 'Problematic Paper Screener' automatically selects suspect publications for post-publication (re)assessment. *arXiv:2210.04895*, 2022.

[19] M. Sharma, M. Tong, T. Korbak, et al. Towards understanding sycophancy in language models. *arXiv:2310.13548*, 2023.

[20] Y. Bai, S. Kadavath, S. Kundu, et al. Constitutional AI: Harmlessness from AI feedback. *arXiv:2212.08073*, 2022.